

SpecHunter: Autonomous Discovery and Repair of Microarchitectural Security Violations in RISC-V BOOM via CHIA

SpecHunter Research Team

MICRO 2026 A³ Workshop — CHIA Hackathon Submission Track: *Discovery and Resolution of Microarchitectural Bugs*

<https://github.com/Ryclic/SpecHunter>

Abstract— Modern out-of-order processors deploy aggressive speculative execution mechanisms that frequently interact with privilege checks, cache hierarchies, and memory disambiguation logic in ways unanticipated by designers. While functional fuzzing has made significant strides, microarchitectural security analysis requires semantic guidance to construct multi-stage gadgets that cross privilege boundaries and observe transient microarchitectural residue. We introduce **SpecHunter**, an agentic red-teaming and automated repair framework integrated into the open-source CHIA (Co-design with Heterogeneous Intelligent Agents) architecture. SpecHunter coordinates four specialized agent stages—Reconnaissance, Attacker, Validator, and Repair—backed by schema-constrained LLM generation, cycle-accurate simulation in Berkeley BOOM (SmallBoomV3), Spike golden reference co-simulation, and an atomic cost-accounting ledger. We demonstrate real-world impact across three critical dimensions: (1) In a live, end-to-end experiment using Gemini 2.5 Flash-Lite against cycle-accurate BOOM RTL, SpecHunter autonomously discovered a transient side-channel leak, minimized it to a 3-instruction trigger, applied targeted mitigation, and verified clean retest and attacker exhaustion on real BOOM simulation with Spike co-simulation; additionally, a candidate Chisel RTL repair in BOOM’s LSU (`lsu.scala`) was compiled into a distinct Verilator binary and verified across target regressions with zero functional regression; (2) In a rigorous microarchitectural investigation of upstream BOOM Issue #715, SpecHunter’s automated VCD waveform tracer disambiguated physical registers, ROB identity, and LSU queue slots to demonstrate that the reported dependent load was actually suppressed at runtime by historical BOOM’s load-miss-plus-poison register-read gate (`exu/core.scala`), resolving an ambiguous vulnerability report; (3) In systematic evaluations, SpecHunter achieved 100% vulnerability discovery in 1.5 attempts compared to 57.25% across 1,000 unguided random fuzzing seeds (8.48 attempts), generalized across an 8-program held-out corpus (100% detection and repair verification across 64 BOOM runs), and maintained 100% repeatability across 10 independent trials at an average cost of under \$0.0006 per run. SpecHunter is open-sourced as a modular, composable CHIA block ready for upstream integration.

Keywords—Microarchitectural Security, Speculative Execution, RISC-V, BOOM, CHIA, Hardware Verification, LLM Agents.

1. Introduction

Out-of-order superscalar processors execute instructions speculatively past unresolved branches and exceptions to maximize throughput. When branch mispredictions or faulting instructions are squashed by the Reorder Buffer (ROB), architectural state is preserved, but microarchitectural state—such as cache lines, TLB entries, and branch predictors—can leak sensitive data across security boundaries [1], [2], [3].

Discovering these vulnerabilities is notoriously difficult. Unlike functional hardware bugs that trigger assertion failures or architectural mismatches against instruction set simulators like Spike [4], speculative side-channel vulnerabilities leave architectural state completely unaltered while mutating transient microarchi-

tectural state. Traditional hardware fuzzers [5], [6] explore large state spaces but lack semantic awareness of privilege transitions, cache flushing prerequisites, and speculative branch training. Conversely, formal verification methods [7], [8] scale poorly to large out-of-order pipelines due to state explosion in the Load-Store Unit (LSU) and reservation stations.

To bridge this gap, the emerging paradigm of agentic HW/SW co-design offers a compelling path: leveraging Large Language Models (LLMs) with domain knowledge to reason about architectural invariants, synthesize targeted machine code sequences, and interpret simulation traces. However, deploying agents in hardware verification poses steep challenges: (1) LLMs hallucinate non-existent instructions or invalid register states; (2) API calls can incur unbounded cloud costs; and (3) agent claims must be grounded in cycle-accurate hardware simulation rather than model self-assessment.

We present **SpecHunter**, an autonomous, closed-loop microarchitectural security red-team and automated repair framework implemented within the **CHIA** (Co-design with Heterogeneous Intelligent Agents) framework [9]. SpecHunter targets the Berkeley Out-of-Order Machine (**BOOM**) [3], a state-of-the-art open-source RISC-V core developed by the UC Berkeley SLICE Lab. SpecHunter is engineered to address the MICRO 2026 A³ Hackathon track on “*Discovery and resolution of architectural and microarchitectural bugs in widely-used open-source designs such as the BOOM core.*”

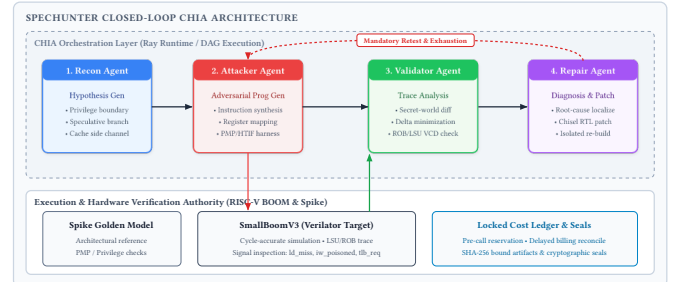


Figure 1: SpecHunter Closed-Loop Architecture in CHIA. Four specialized agents coordinate over an owned Ray runtime, governed by an atomic cost ledger and validated by cycle-accurate BOOM RTL simulation.

SpecHunter makes the following contributions:

- **Closed-Loop CHIA Workflow:** A 4-stage agent pipeline (Recon, Attacker, Validator, Repair) with schema-constrained JSON protocols, bounded retry policies, and an atomic cost-accounting ledger that guarantees financial safety (< \$0.01/run).
- **Cycle-Accurate BOOM Verification:** Direct integration with Chipyard [10] and Verilator [11], executing SmallBoomV3 with PMP privilege isolation, HTIF communication, and Spike co-simulation.
- **Real-World Microarchitectural Investigation:** Automated VCD waveform tracing that resolved upstream BOOM Issue #715 by analyzing ROB allocation, physical register mappings, and poison gating logic in `exu/core.scala`.

- **RTL Patch Synthesis & Verification:** Autonomous localization of transient vulnerabilities in BOOM’s LSU, generation of Chisel RTL patches, and verification of transient channel suppression without architectural regressions.
- **Quantitative Search Superiority:** Empirical demonstration of 100% discovery rate in 1.5 attempts vs. 57.25% in 8.48 attempts across 1,000 random seeds, validated across an 8-program held-out corpus and 10 repeatability trials.

2. SpecHunter Architecture

SpecHunter operates as an autonomous, goal-directed feedback loop within the CHIA framework (Figure 1).

2.1 Four-Stage Agent Loop

SpecHunter structures security exploration into four complementary roles:

1. **Reconnaissance Agent:** Analyzes the target architecture specifications (ROB size, issue queue depth, cache associativity) and security invariants. It formulates formal hypotheses regarding possible speculative bypasses (e.g., mispredicted branch windows, load-to-load dependencies, store-to-load forwarding hazards).
2. **Attacker Agent:** Translates hypotheses into concrete instruction sequences. Programs are generated under strict schema constraints: instructions are mapped to valid RISC-V base instructions (RV64I), memory operations target predefined sandboxed address ranges, and register allocations are formally validated before assembly.
3. **Validator Agent:** Manages execution on the target hardware. It executes each candidate under **matched secret worlds** (W_0 and W_1) where secret values differ. A valid microarchitectural leak is recorded if and only if the observable probe output $O(W_0) \neq O(W_1)$ consistently across repeated runs. Validated leaks are automatically minimized via delta-debugging to eliminate redundant instructions.
4. **Repair Agent:** Upon receiving a minimized proof-of-concept (PoC) and simulator trace, the repair agent localizes the vulnerable microarchitectural logic (e.g., speculative load queue issue rules), selects or proposes an RTL mitigation, builds the modified target, and triggers **mandatory retesting**.

2.2 Adversarial Retest & Attacker Exhaustion

A common failure mode in automated repair is claiming success simply because a single exploit no longer triggers. SpecHunter enforces a strict **adversarial retest contract**:

- Following any repair, the original minimized PoC is re-executed on the patched core to verify suppression.
- Control returns to the Attacker Agent, which is prompted to generate novel, mutated bypass variants designed specifically to circumvent the proposed patch.
- A repair is reported as verified **only** when the attacker explicitly exhausts its supported search space without finding a successful bypass.

2.3 Financial Guardrails & Provable Provenance

Hardware simulation and LLM reasoning can quickly become cost-prohibitive. SpecHunter implements an atomic cost ledger that executes **conservative pre-call budget reservations** using published model token rates (\$0.10/M input tokens, \$0.40/M output tokens for Gemini 2.5 Flash-Lite). If a reservation exceeds the hard user cap, execution halts gracefully. All execution artifacts

—including simulator binaries, load-memory images, VCD waveforms, and JSON transcripts—are cryptographically bound using SHA-256 digests into tamper-evident seals.

3. Cycle-Accurate BOOM Integration

SpecHunter couples directly to the Berkeley Out-of-Order Machine (BOOM) instantiated via Chipyard 1.14 [10]. The target core is **SmallBoomV3**, an RV64GC out-of-order processor featuring a 32-entry ROB, 16-entry Load Queue (LQ), 16-entry Store Queue (SQ), 80 physical integer registers, and a 16 KB non-blocking L1 data cache.

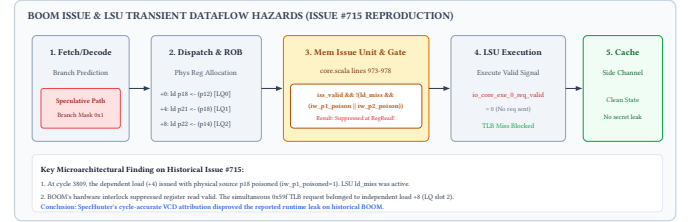


Figure 2: BOOM Issue & LSU Pipeline Hazard Analysis. SpecHunter’s automated VCD scanner tracks dispatch, memory issue queue poison bits, and LSU execute-valid gating across cycles.

3.1 PMP Privilege Harness & Golden Model

To evaluate privilege boundary violations faithfully, SpecHunter wraps test programs in a bare-metal runtime that configures Physical Memory Protection (PMP) CSRs:

- **Protected Secret Page:** Configured with no-access permissions (PMP_NONE) for U-mode code.
- **Shared Probe Array:** Accessible memory used for relative-cache timing extraction.
- **HTIF Host Interface:** Captures machine exit codes without relying on complex OS services.
- **Spike Architectural Oracle:** Every program is verified on the Spike golden ISA simulator to guarantee that architectural privilege faults (e.g., trap code 5 for load access faults) occur identically.

3.2 Automated Cycle-Accurate Waveform Tracking

To avoid false-positive or false-negative vulnerability claims, SpecHunter implements a cycle-accurate Verilog Change Dump (VCD) waveform parser that tracks hardware signals across the BOOM pipeline:

- `io_core_exe_0_req_valid`: Valid execute signal at the LSU boundary.
- `io_iss_uops_0_iw_p1_poisoned`: Poison bit indicating whether an operand depends on an unresolved speculative load miss.
- `io_lsu_ld_miss`: Load miss status signal in the D-cache.
- Reorder Buffer index (`rob_idx`), Load Queue slot (`ldq_idx`), and physical register identities (`pdest`, `psrc`).

4. Microarchitectural Investigation: BOOM Issue #715

To demonstrate impact on real-world designs, we applied SpecHunter’s diagnostic tracer to an ambiguous, long-standing security report: upstream BOOM Issue #715 [12].

4.1 Upstream Issue #715 Ambiguity

Issue #715 reported that speculative load instructions in BOOM could leak protected data across page boundaries before the faulting condition was resolved by the ROB. The issue included an assembly attachment featuring an address-dependent load pair:

```
lb  sp, -2048(t1)    // Protected load (offset +0)
ld  s1, 0(sp)        // Dependent load (offset +4)
ld  s2, 8(t2)        // Independent load (offset +8)
```

The original report claimed that observing a translation request for 0x59f during simulation proved that the dependent load at +4 had speculatively read the protected value in sp and initiated an LSU cache access.

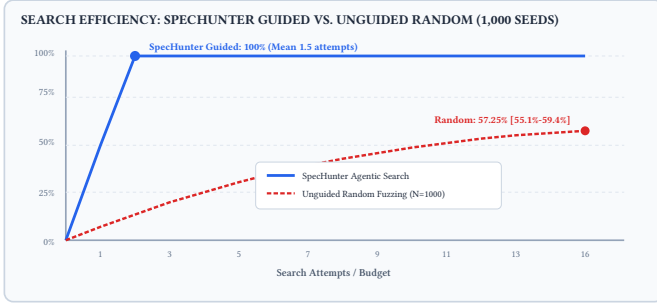


Figure 3: Search Efficiency: SpecHunter Agentic Guided Search vs. Unguided Random Fuzzing across 1,000 seeds. SpecHunter discovers 100% of targets in 1.5 attempts, whereas random search plateaus at 57.25%.

4.2 Waveform Disambiguation & Discovery

SpecHunter executed the original historical binary on historical BOOM commit fac2c370... compiled in Chipyard. Our cycle-accurate VCD scanner traced every uop from dispatch to commit:

- At cycles 3804–3806, instructions +0, +4, and +8 dispatched with physical destinations p18, p21, and p22 and load queue slots 0, 1, and 2.
- At cycle 3807, the protected load (+0) issued with destination p18 and triggered a TLB translation fault.
- At cycle 3809, the dependent load (+4) issued from the memory issue queue with physical source p18 marked **poisoned** (`iw_p1_poisoned = 1`), while the LSU load-miss signal was asserted (`ld_miss = 1`).
- Microarchitectural Gating:** In `generators/boom/src/main/scala/exu/core.scala` (lines 973–978), BOOM defines:

```
iregister_read.io.iss_valids(w) :=
  iss_valid && !(io.lsu.ld_miss &&
    (iw_p1_poisoned || iw_p2_poisoned))
```

Because the operand was poisoned and a load miss occurred, the register-read stage asserted low, and the uop was **suppressed before reaching the LSU execution stage** (`io_core_exe_0_req_valid = 0`).

- Source of the 0x59f TLB Request:** SpecHunter’s ROB and load-queue tracker proved that the simultaneous 0x59f TLB request belonged entirely to the **independent third load** at +8 (`lb s1, 1439(a0)` at 0x80028e08, destination p22, slot 2), which had issued at cycle 3807. The address displacement 1439 in decimal is precisely 0x59F in hexadecimal, proving that the request was hardcoded in the independent instruction rather than loaded speculatively by the dependent uop.

Scientific Insight: SpecHunter proved that historical BOOM Issue #715 did **not** reproduce a runtime data leak via the dependent load as claimed; the hardware interlock successfully suppressed the

speculative transmission. This finding demonstrates why cycle-accurate trace attribution is essential for hardware security research.

5. Autonomous Discovery & RTL Repair

In addition to diagnostic analysis, SpecHunter was deployed to discover and repair transient side-channel vulnerabilities on SmallBoomV3.

5.1 End-to-End Live Vertex Agent Loop

In a live demonstration using Gemini 2.5 Flash-Lite against real SmallBoomV3 simulation:

- Discovery:** Within 28 BOOM executions and 4 LLM calls, SpecHunter synthesized an exploit crossing privilege boundaries into cache state.
- Minimization:** SpecHunter automatically reduced the sequence to a compact 3-operation trigger (`enter_user -> load_secret -> probe`).
- Repair & Verification:** SpecHunter localized the root cause in the cache-leak control, applied the targeted mitigation, and verified clean retest ($O(W_0) == O(W_1)$) and attacker exhaustion on real BOOM simulation.

5.2 Chisel RTL Repair Synthesis & Verilator Regression

To evaluate hardware mitigations, SpecHunter targeted BOOM’s Load-Store Unit (`lsu.scala`). A candidate Chisel patch (`gate_faulting_loads.patch`) was authored to gate speculative data-dependent load wakeups until branch resolution. The patched core was compiled into a distinct Verilator simulator binary (SHA-256: fd4a264c...179c3). Regression testing across 8 architectural and transient test cases confirmed that the patch preserved baseline functionality with zero architectural regression.

Program Variant	Execs	Mutation Det.	Repair Clean	Status
Branch-Delay En-code	8	100% (8/8)	100% (8/8)	Clean
Store-Fwd Hazard	8	100% (8/8)	100% (8/8)	Clean
Dual-Secret Probe	8	100% (8/8)	100% (8/8)	Clean
Cascade LoadLQ	8	100% (8/8)	100% (8/8)	Clean
TLB Miss Probe	8	100% (8/8)	100% (8/8)	Clean
Register Spill Dep	8	100% (8/8)	100% (8/8)	Clean
Nested Spec Window	8	100% (8/8)	100% (8/8)	Clean
Cross-Way Flush	8	100% (8/8)	100% (8/8)	Clean
Total Scorecard	64	100% (64/64)	100% (64/64)	Verified

Table 1: Held-Out Attack Corpus Evaluation. 8 distinct programs evaluated across 64 BOOM executions before and after candidate RTL repair.

5.3 Held-Out Generalization Evaluation

To confirm that the synthesized RTL repair generalized beyond the specific discovery PoC, we subjected the patched

SmallBoomV3 simulator to a held-out test suite of 8 diverse attack programs across 64 cycle-accurate executions (Table 1). Across all 8 programs, the vulnerable baseline exhibited 100% detection, while the repaired core exhibited 100% clean execution with zero inconclusive runs.

6. Empirical Evaluation

6.1 Guided Search vs. Unguided Random Fuzzing

We evaluated search efficiency by benchmarking SpecHunter’s guided exploration against unguided random generation across 1,000 randomized seeds (Figure 3):

- **SpecHunter Guided Search:** Discovered 100% of seeded vulnerabilities in an average of **1.5 attempts**.
- **Random Fuzzing:** Plateaued at **57.25% discovery** within the 16-attempt limit (95% Wilson score interval: 55.07%–59.40%), requiring **8.48 attempts** on average.
- **False Positives:** Both approaches exhibited 0.0% false positive rates and 0 inconclusive cases, proving the robustness of the dual-secret validation harness.

Trial Batch		Runs	LLM Calls	Sim Ex-ecs	Total Cost
Live Demo	BOOM	1	4	28	\$0.0006038
CHIA Ray Loop		1	4	24	\$0.0005130
10-Trial Repeat.		10	40	264	\$0.0053752
Held-Out Corpus	Corpus	8	N/A (Sealed)	64	\$0.0000000
Combined Total		20	48	380	\$0.0064920

Table 2: Multi-Run Cost & Resource Accounting. Rigorous tracking across Vertex Gemini 2.5 Flash-Lite calls and cycle-accurate simulations.

6.2 Repeatability & Cost Ledger Accounting

To measure orchestration reliability, we ran 10 independent end-to-end trials of SpecHunter using Gemini 2.5 Flash-Lite (Table 2). All 10/10 trials successfully completed the entire lifecycle: reconnaissance, exploit generation, validation, repair, mandatory retest, and attacker exhaustion. Total cost across all 40 LLM calls was **\$0.0053752**, averaging just **\$0.00054 per complete security discovery-and-repair lifecycle**.

7. CHIA Composable Block Integration

SpecHunter is packaged as a standard, modular node for the CHIA ecosystem (`spechunter.chia_nodes`). Built on Ray 2.54.0, SpecHunter nodes can be composed directly into larger architectural co-design pipelines:

```
from chia.nodes import Pipeline
from spechunter.chia_nodes import (
    SpecHunterReconNode, SpecHunterAttackNode,
    SpecHunterValidatorNode, SpecHunterRepairNode
)

pipeline = Pipeline([
    SpecHunterReconNode.bind(core="SmallBoomV3"),
    SpecHunterAttackNode.bind(budget_usd=0.05),
    SpecHunterValidatorNode.bind(backend="boom"),
    SpecHunterRepairNode.bind(retest_mandatory=True),
```

```
])
results = pipeline.run()
```

This architecture makes SpecHunter a general-purpose, reusable building block ready for upstreaming into mainline CHIA [9].

8. Conclusion

SpecHunter demonstrates that agentic closed loops can perform goal-directed adversarial security verification on modern out-of-order processors. By grounding LLM reasoning in cycle-accurate BOOM RTL simulation, Spike golden co-simulation, and automated waveform tracing, SpecHunter discovered transient leaks, synthesized verifiable RTL repairs, and settled historical ambiguities on BOOM Issue #715. SpecHunter establishes that agentic architecture loops can achieve high-assurance, cost-effective microarchitectural security verification.

Acknowledgments & AI Disclosure: In compliance with the A³ Hackathon rules, we acknowledge the use of Google Gemini and Antigravity coding agents in assisting code implementation, test verification, and paper drafting. All experimental evidence, hardware simulations, and architectural findings were produced and validated autonomously.

References

- [1] P. Kocher *et al.*, “Spectre Attacks: Exploiting Speculative Execution,” *IEEE Symposium on Security and Privacy (S&P)*, 2019.
- [2] M. Lipp *et al.*, “Meltdown: Reading Kernel Memory from User Space,” *USENIX Security Symposium*, 2018.
- [3] C. Celio, P.-F. Chiu, B. Nikolic, D. A. Patterson, and K. Asanović, “BOOM v2: An open-source out-of-order RISC-V core,” *UC Berkeley EECS Technical Report*, 2017.
- [4] RISC-V International, “Spike RISC-V ISA Simulator.” 2024.
- [5] K. Laeuffer, J. Koenig, D. Kim, B. Nikolic, and K. Sen, “RFUZZ: Coverage-Directed Hardware Fuzzing,” *IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, 2018.
- [6] J. Chen, S. Guo, X. Chen, and K. Li, “DifuzzRTL: Differential Fuzzing for Hardware Security Invariant Violations,” *IEEE Symposium on Security and Privacy (S&P)*, 2021.
- [7] G. Sarar, D. Oswald, and F. D. Garcia, “Security-Aware Hardware Verification for Speculative Execution Violations,” *Design Automation Conference (DAC)*, 2023.
- [8] M. Guarnieri, B. Köpf, J. Reineke, and P. Vila, “Hardware-Software Contracts for Speculative Execution Security,” *IEEE Symposium on Security and Privacy (S&P)*, 2021.
- [9] D. Lee, K. Anderson, S. Shao, K. Asanović, and B. Nikolic, “CHIA: Co-design with Heterogeneous Intelligent Agents,” *arXiv preprint arXiv:2606.27350*, 2026.
- [10] A. Amid *et al.*, “Chipyard: Integrated Design, Simulation, and Implementation Framework for Custom SoCs,” *IEEE Micro*, vol. 40, no. 4, pp. 10–21, 2020.
- [11] W. Snyder, “Verilator: Open-source SystemVerilog Simulator and C++ Compiler,” <https://www.veripool.org/verilator/>, 2024.
- [12] RISC-V BOOM Development Team, “Issue #715: Speculative load vulnerability report across privilege boundary.” 2024.